

Graph DSA Handbook

A Complete, Practical Guide to Graph Algorithms in Python

PART 1: GRAPH FUNDAMENTALS

- 1.1 What is a Graph?
- 1.2 Core Terminology
- 1.3 Types of Graphs
- 1.4 Ways to Represent a Graph
- 1.5 How to Read This Handbook

PART 2: ALGORITHMS

- 2.1 Breadth-First Search (BFS)
 - Practice Questions — BFS
 - 2.2 Depth-First Search (DFS)
 - Practice Questions — DFS
 - 2.3 Topological Sort — DFS Based
 - Practice Questions — Topological Sort (DFS)
 - 2.4 Topological Sort — BFS Based (Kahn's Algorithm)
 - Practice Questions — Topological Sort (Kahn's / BFS)
 - 2.5 Minimum Spanning Tree (MST) — Concept
 - 2.6 Prim's Algorithm
 - Practice Questions — Prim's Algorithm
 - 2.7 Kruskal's Algorithm
 - Practice Questions — Kruskal's Algorithm
 - 2.8 Union-Find (Disjoint Set Union - DSU)
 - Practice Questions — Union-Find
 - 2.9 Dijkstra's Algorithm
 - Practice Questions — Dijkstra's Algorithm
 - 2.10 Bellman-Ford Algorithm
 - Practice Questions — Bellman-Ford Algorithm
 - 2.11 Floyd-Warshall Algorithm
 - Practice Questions — Floyd-Warshall Algorithm
- ## PART 3: QUICK REFERENCE CHEAT SHEET
- Decision Guide — "Which Algorithm Do I Use?"

PART 1: GRAPH FUNDAMENTALS

1.1 What is a Graph?

A **graph** is a collection of **nodes (vertices)** connected by **edges**. It represents *relationships* between things — cities connected by roads, people connected by friendships, computers connected by network links, tasks connected by dependencies.

Graph $G = (V, E)$ where V = set of vertices, E = set of edges.

1.2 Core Terminology

Term	Meaning
Vertex (Node)	A single point/entity in the graph
Edge	A connection between two vertices
Path	A sequence of vertices connected by edges, no repeated vertex (usually)
Simple Path	A path with no repeated vertices
Walk	Sequence of vertices/edges where repetition is allowed
Cycle	A path that starts and ends at the same vertex
Degree	Number of edges attached to a vertex
In-Degree	(Directed graph) number of edges coming INTO a vertex
Out-Degree	(Directed graph) number of edges going OUT of a vertex
Adjacent Vertices	Two vertices connected directly by an edge
Connected Component	A group of vertices all reachable from each other
Weight	A cost/value assigned to an edge
Self-loop	An edge from a vertex to itself
Parallel/Multi Edge	More than one edge between the same pair of vertices

1.3 Types of Graphs

By Direction

- **Undirected Graph** — edges have no direction. $A-B$ means A reaches B and B reaches A .
- **Directed Graph (Digraph)** — edges have direction. $A \rightarrow B$ does NOT imply $B \rightarrow A$.

By Weight

- **Unweighted Graph** — all edges are considered equal cost (cost = 1).
- **Weighted Graph** — each edge has a cost/weight (distance, time, money, etc.)

By Cycles

- **Cyclic Graph** — contains at least one cycle (path that returns to start).
- **Acyclic Graph** — no cycles at all.
- **DAG (Directed Acyclic Graph)** — directed + no cycles. Used for scheduling, dependency resolution, build systems.

By Connectivity

- **Connected Graph** (undirected) — every vertex reachable from every other vertex.
- **Disconnected Graph** — graph has 2+ separate components.
- **Strongly Connected** (directed) — every vertex reachable from every other vertex following edge direction.

- **Weakly Connected** (directed) — connected only if you ignore edge direction.

By Density

- **Sparse Graph** — few edges relative to vertices ($E \approx V$). Use Adjacency List.
- **Dense Graph** — edges close to max possible ($E \approx V^2$). Adjacency Matrix is fine.

Special Structures

- **Tree** — a connected, undirected, acyclic graph. V vertices, exactly $V-1$ edges. Exactly one path between any two nodes.
- **Forest** — a disjoint collection of trees (acyclic, possibly disconnected).
- **Bipartite Graph** — vertices can be split into 2 sets such that every edge connects a vertex from one set to the other (no edge within the same set). Useful for matching problems.
- **Complete Graph** — every pair of vertices is connected. $E = V(V-1)/2$ for undirected.
- **Multigraph** — allows multiple edges between the same pair of vertices, and/or self-loops.

1.4 Ways to Represent a Graph

1. Adjacency Matrix

A $V \times V$ 2D array. `matrix[i][j] = 1` (or weight) if edge exists between `i` and `j`, else `0/∞`.

```
# Adjacency Matrix for a graph with V vertices
V = 5
adj_matrix = [[0] * V for _ in range(V)]

def add_edge_matrix(u, v, weight=1, directed=False):
    adj_matrix[u][v] = weight
    if not directed:
        adj_matrix[v][u] = weight # mirror for undirected
```

- **Space:** $O(V^2)$ — wasteful for sparse graphs.
- **Check if edge (u,v) exists:** $O(1)$ — very fast.
- **Iterate all neighbors of u:** $O(V)$ — slow even if `u` has few neighbors.
- **Best for:** dense graphs, small V , frequent edge-existence checks (e.g. Floyd-Warshall).

2. Adjacency List

Each vertex stores a list of its neighbors. Most commonly used in interviews/competitive programming.

```
from collections import defaultdict

adj_list = defaultdict(list)

def add_edge_list(u, v, weight=1, directed=False):
    adj_list[u].append((v, weight)) # store (neighbor, weight)
    if not directed:
        adj_list[v].append((u, weight)) # mirror for undirected
```

- **Space:** $O(V + E)$ — efficient, only stores real edges.
- **Check if edge (u,v) exists:** $O(\text{degree}(u))$ — slower than matrix.
- **Iterate all neighbors of u:** $O(\text{degree}(u))$ — fast, only real neighbors.
- **Best for:** sparse graphs (most real-world graphs), BFS/DFS, almost all interview problems.

3. Edge List

A flat list of all edges `(u, v, weight)`. No per-vertex structure.

```
edge_list = []

def add_edge_list_form(u, v, weight=1):
    edge_list.append((u, v, weight))
```

- **Space:** $O(E)$.
- **Best for:** algorithms that process all edges directly and don't need neighbor lookup — e.g. **Kruskal's Algorithm**,

Bellman-Ford.

- **Weak at:** finding neighbors of a specific vertex (must scan all edges).

Quick Comparison

Representation	Space	Edge Check	Get Neighbors	Best Use
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$	Dense graphs, Floyd-Warshall
Adjacency List	$O(V+E)$	$O(\text{deg})$	$O(\text{deg})$	Most problems, sparse graphs
Edge List	$O(E)$	$O(E)$	$O(E)$	Kruskal, Bellman-Ford

1.5 How to Read This Handbook

For every algorithm below you will get, in this exact order:

1. **What it's for** — the plain-English purpose.
2. **Use Cases** — real scenarios where it's applied.
3. **Limitations** — where/why it breaks or performs badly.
4. **Time & Space Complexity.**
5. **Approach (in short)** — the “what I actually do” mental model.
6. **Keywords & Constraints** — trigger phrases in problems + typical constraint sizes that hint you should use this algorithm.
7. **Fully commented Python code.**
8. **10 Practice Questions** — each with a short description and a one-line reasoning of *why this algorithm fits* (and often, why simpler algorithms fail).

PART 2: ALGORITHMS

2.1 Breadth-First Search (BFS)

What it's for: Explores a graph level-by-level (layer by layer) starting from a source node, using a queue. It's the go-to algorithm for finding the **shortest path in an unweighted graph** and for exploring nodes in order of distance from the source.

Use Cases: - Shortest path / minimum number of edges in an unweighted graph. - Level-order exploration (social network "friends at distance k"). - Finding connected components. - Multi-source shortest distance (rotten oranges style problems). - Bipartite graph checking (via 2-coloring during BFS). - Web crawlers, peer-to-peer networks (nearest node discovery).

Limitations: - Does NOT work correctly for shortest path if the graph is **weighted** (use Dijkstra instead). - Uses more memory than DFS for wide/bushy graphs (stores an entire frontier level in the queue). - Not suitable for problems where you need to explore "as deep as possible first" (e.g., backtracking, path existence with all combinations) — DFS is better there.

Time Complexity: $O(V + E)$ — every vertex and edge visited once. **Space Complexity:** $O(V)$ — for the queue and visited set.

Approach (short): Push source into a queue → mark visited → pop front → push all unvisited neighbors → repeat until queue empty. Think "ripple in water" — explore in expanding circles.

Keywords & Constraints: - "shortest path", "minimum steps", "minimum number of moves", "level order", "nearest", "unweighted graph" - Typical constraints: $V, E \leq 10^5$ to 10^6 (linear algorithm, so large inputs are fine) - Grid problems: "minimum steps to reach", "shortest path in binary matrix"

```
from collections import deque

def bfs(adj, source, V):
    """
    adj: adjacency list, adj[u] = list of neighbors
    source: starting node
    V: total number of vertices
    Returns: list of distances from source (in terms of number of edges)
    """
    visited = [False] * V          # track visited nodes so we don't reprocess them
    distance = [-1] * V            # -1 means unreachable; will hold min edges from source

    queue = deque([source])        # start BFS from source
    visited[source] = True         # mark source visited immediately (not on pop!)
    distance[source] = 0           # distance to itself is 0

    while queue:
        node = queue.popleft()     # FIFO -> ensures level-by-level exploration

        for neighbor in adj[node]:
            if not visited[neighbor]:
                visited[neighbor] = True          # mark visited when ENQUEUED,
                                                  # not when dequeued, to avoid
                                                  # adding the same node twice
                distance[neighbor] = distance[node] + 1 # one more edge than parent
                queue.append(neighbor)

    return distance
```

Practice Questions — BFS

Q1. Shortest Path in Unweighted Graph Given an unweighted graph and a source, find the minimum number of edges to reach every other node. *Approach:* Standard BFS — since every edge has equal weight (1), the first time BFS reaches a node is the shortest path. DFS wouldn't guarantee shortest path because it can reach a node via a long route first.

Q2. Rotten Oranges Grid has fresh oranges, rotten oranges, and empty cells. Rotten oranges rot adjacent fresh ones each minute. Find minimum time until no fresh orange remains. *Approach:* Multi-source BFS — push ALL initially rotten oranges into the queue at once (they're all "distance 0"), then BFS layer by layer; each layer = 1 minute.

Q3. Word Ladder Transform start word to end word changing 1 letter at a time, each intermediate word must exist in a dictionary. Find shortest transformation length. *Approach:* BFS on implicit graph where each word is a node and edges connect words differing by 1 letter — since we want the *shortest* sequence, BFS beats DFS which would just find a path, not the shortest.

Q4. Nearest Exit from Entrance in Maze Find shortest distance from entrance to any boundary cell that isn't a wall. *Approach:* BFS from the entrance cell — grid shortest path (unweighted) always signals BFS.

Q5. Binary Matrix Shortest Clear Path Find shortest path from top-left to bottom-right in a binary grid (0 = open, 1 =

blocked), moving in 8 directions. *Approach*: BFS again — unweighted grid, so first arrival = shortest.

Q6. Is Graph Bipartite? Check whether graph vertices can be 2-colored such that no adjacent vertices share a color. *Approach*: BFS with coloring — color neighbors the opposite color of current node; if a conflict is found (neighbor already same color), graph isn't bipartite. BFS/DFS both work; BFS is often cleaner for level-based coloring.

Q7. Number of Islands Count connected groups of '1's in a grid (4-directionally connected). *Approach*: BFS (or DFS) from every unvisited land cell to mark the entire island as visited, incrementing island count once per BFS call.

Q8. Snake and Ladder Problem Given a board with snakes/ladders, find minimum number of dice throws to reach the last cell. *Approach*: BFS on a graph where each cell connects to up to 6 next cells (dice roll outcomes) — minimum throws = shortest path in an unweighted graph.

Q9. Open the Lock 4-digit lock, each move changes 1 wheel by 1 step; certain combinations are “deadends” you can't touch. Find minimum moves to reach target. *Approach*: BFS over the state space (each state = a 4-digit string), since we want the minimum number of moves — a classic unweighted shortest-path-in-implicit-graph problem.

Q10. Level Order Traversal Distance (Friend Circles at Distance K) Given a social network graph and a person, find all friends exactly K connections away. *Approach*: BFS, stop expanding after exactly K levels — BFS naturally processes nodes level by level, making “exact distance K” queries trivial.

2.2 Depth-First Search (DFS)

What it's for: Explores a graph by going as deep as possible along one branch before backtracking. Implemented via recursion or an explicit stack. It's the foundation for cycle detection, topological sort, connected components, and path-existence problems.

Use Cases: - Detecting cycles in directed/undirected graphs. - Topological sorting (DFS-based). - Finding connected components / strongly connected components (Tarjan's, Kosaraju's). - Path existence (does a path exist from A to B). - Solving mazes, backtracking-style search (Sudoku, N-Queens use DFS-like recursion). - Counting islands, flood fill.

Limitations: - Does NOT find the shortest path in an unweighted graph (BFS does). - Can hit **recursion depth limits** in Python for very deep/large graphs (need to increase `sys.setrecursionlimit` or use iterative DFS with an explicit stack). - Can be slower in practice than BFS for "find nearest" type queries because it explores deep, irrelevant branches first.

Time Complexity: $O(V + E)$. **Space Complexity:** $O(V)$ — recursion stack / explicit stack + visited array.

Approach (short): Start at a node → mark visited → recursively visit an unvisited neighbor → backtrack when no unvisited neighbors remain. Think "go as far as you can, then come back."

Keywords & Constraints: - "connected components", "cycle detection", "all paths", "backtracking", "reachability", "flood fill" - Constraints: $V, E \leq 10^5$ - 10^6 typically fine (linear); watch for recursion depth on $V > 10^4$ in Python — prefer iterative DFS.

```
def dfs_recursive(adj, node, visited, result):
    """
    adj: adjacency list
    node: current node being explored
    visited: set/array tracking visited nodes
    result: order in which nodes were visited (optional, useful for debugging)
    """
    visited[node] = True          # mark as visited the moment we enter it
    result.append(node)          # record visit order

    for neighbor in adj[node]:
        if not visited[neighbor]:
            dfs_recursive(adj, neighbor, visited, result) # go deep first

def dfs_iterative(adj, source, V):
    """
    Iterative version using an explicit stack -- avoids Python recursion limits
    for very deep/large graphs.
    """
    visited = [False] * V
    stack = [source]
    result = []

    while stack:
        node = stack.pop()        # LIFO -> depth first behaviour

        if visited[node]:
            continue              # skip if already processed (can be pushed twice)

        visited[node] = True
        result.append(node)

        for neighbor in adj[node]:
            if not visited[neighbor]:
                stack.append(neighbor) # push to explore later, deepest first

    return result
```

Practice Questions — DFS

Q1. Number of Connected Components Count how many separate connected groups exist in an undirected graph. *Approach:* Loop through all vertices; run DFS from every unvisited vertex, incrementing a counter each time — each DFS call fully explores one component.

Q2. Detect Cycle in Undirected Graph Check if an undirected graph has a cycle. *Approach:* DFS while tracking the parent node; if you reach an already-visited node that is NOT the immediate parent, a cycle exists.

Q3. Detect Cycle in Directed Graph Check if a directed graph has a cycle. *Approach:* DFS with a recursion-stack (path) tracker; if you reach a node that's already in the current recursion path (not just visited overall), it's a cycle — this distinguishes it from the undirected version.

Q4. Path Exists Between Two Nodes Given source and destination, check if any path connects them. *Approach:* Simple DFS from source, stop early if destination found — BFS works too, but DFS uses less bookkeeping when you only need existence, not shortest length.

Q5. All Paths From Source to Target (DAG) Print/return all possible paths between two nodes in a DAG. *Approach:* DFS with backtracking — build a path list, recurse into neighbors, and pop the last node when backtracking; DFS naturally explores every branch, which is exactly what “all paths” needs (BFS can’t easily enumerate full paths).

Q6. Flood Fill / Number of Islands Fill or count connected regions of the same value in a grid. *Approach:* DFS from each unvisited cell, recursively visiting all 4/8-directionally connected same-value cells and marking them visited.

Q7. Course Schedule (Cycle Check for Prerequisites) Given course prerequisites as directed edges, determine if it’s possible to finish all courses. *Approach:* DFS cycle detection in a directed graph — if there’s a cycle in prerequisites, it’s impossible (you’d need to finish a course before itself).

Q8. Clone a Graph Deep copy a graph given a reference node. *Approach:* DFS while maintaining a hashmap of `original_node -> cloned_node` to avoid infinite loops/re-cloning on cycles.

Q9. Count Strongly Connected Components (Kosaraju’s Algorithm) Find groups of nodes in a directed graph where every node can reach every other node in the group. *Approach:* Two-pass DFS — first DFS to get finish-order (stack), reverse the graph, then DFS again in that stack order; DFS’s ability to track finishing times is essential here, BFS can’t do this.

Q10. Word Search in Grid Given a grid of letters and a word, check if the word can be constructed by moving to adjacent cells (no cell reused). *Approach:* DFS with backtracking from each matching starting cell, marking cells visited temporarily and un-marking on backtrack — this is the classic “explore all possibilities” pattern DFS is built for.

2.3 Topological Sort — DFS Based

What it's for: Produces a linear ordering of vertices in a **DAG** such that for every directed edge $u \rightarrow v$, u comes before v in the ordering. Answers “in what order must these tasks happen?”

Use Cases: - Build systems (compile file A before file B). - Course scheduling (finish prerequisite before the course). - Task scheduling with dependencies. - Package/dependency resolution (e.g., npm/pip install order). - Spreadsheet formula evaluation order.

Limitations: - Only works on **DAGs** — a cycle means no valid topological order exists (must detect cycle first). - Doesn't work on undirected graphs (no notion of “before/after”). - If multiple valid orderings exist, DFS-based topo sort gives just ONE of them, not all.

Time Complexity: $O(V + E)$. **Space Complexity:** $O(V)$ — recursion stack + result stack.

Approach (short): Run DFS; when a node has no more unvisited neighbors (fully explored/“finished”), push it onto a stack. At the end, pop the stack — that's your topological order. Think “finish deepest dependencies first, add them to the front.”

Keywords & Constraints: - “prerequisite”, “build order”, “dependency”, “must happen before”, “schedule tasks”, “DAG”
- Constraints: $V, E \leq 10^5$ typical.

```
def topo_sort_dfs(adj, V):
    """
    adj: adjacency list of a DAG
    V: number of vertices
    Returns: a valid topological order (list)
    """
    visited = [False] * V
    stack = [] # will hold nodes in REVERSE finish order

    def dfs(node):
        visited[node] = True
        for neighbor in adj[node]:
            if not visited[neighbor]:
                dfs(neighbor) # fully explore neighbor's subtree first
        stack.append(node)    # push AFTER all neighbors are done
                             # (this node "finishes" last among its subtree)

    for v in range(V):
        if not visited[v]:
            dfs(v) # handle disconnected components too

    stack.reverse() # last finished = should come first
    return stack
```

Practice Questions — Topological Sort (DFS)

Q1. Course Schedule II Return a valid order to finish all courses given prerequisite pairs. *Approach:* DFS-based topo sort directly gives the required order; must first confirm no cycle exists (impossible schedule) using the recursion-stack cycle check.

Q2. Alien Dictionary Given a sorted list of words from an alien language, find the order of characters in that alphabet. *Approach:* Build a graph where edge $a \rightarrow b$ means character a comes before b (derived by comparing adjacent words); topological sort on that character graph gives the alphabet order.

Q3. Build System / Compile Order Given file dependencies, output the order in which files should be compiled. *Approach:* Directed edge $\text{dependency} \rightarrow \text{file}$; topological sort gives compile order — DFS-based version is easy to write recursively for this kind of static dependency graph.

Q4. All Topological Sorts of a DAG Enumerate every valid topological ordering. *Approach:* Backtracking DFS: at each step, pick any node with in-degree 0 among the remaining, temporarily remove it, recurse, then restore — a variation combining DFS with Kahn's in-degree idea to generate ALL orders (not just one).

Q5. Task Scheduling with Dependencies Given tasks and “task A must happen before task B” pairs, output a valid sequence. *Approach:* Same DFS topo sort pattern — model as DAG, run DFS, reverse finish stack.

Q6. Longest Path in a DAG Find the longest path length between any two nodes in a DAG. *Approach:* Topologically sort first, then process nodes in that order doing DP ($\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight})$) — topological order guarantees you process a node only after all its predecessors, which plain DFS/BFS relaxation can't guarantee.

Q7. Minimum Time to Finish All Jobs (with dependencies + durations) Each job has a duration and depends on other jobs; find minimum total time considering parallelism isn't allowed (single machine) or is allowed. *Approach:*

Topological sort first to get a valid processing order, then DP over that order to compute earliest finish times.

Q8. Detect if Course Schedule is Possible Given prerequisites, just answer yes/no whether all courses can be completed. *Approach:* Attempt DFS topo sort while tracking recursion stack; if a cycle is detected during DFS, answer is “No” — this reuses the same DFS traversal as topo sort itself.

Q9. Compilation Order with Multiple Independent Modules Some files have no dependencies at all (disconnected parts of the graph). *Approach:* Loop over all unvisited nodes and DFS each — ensures disconnected DAG components are all included in the final order, a detail beginners often miss.

Q10. Sequence Reconstruction Check if a given sequence is the UNIQUE topological order producible from a set of pairwise constraints. *Approach:* Build the DAG from constraints, and additionally check that at every step there’s only exactly one valid “next” node (in-degree 0) — this nuance is easier to check with Kahn’s BFS version (see next section) than pure DFS, but DFS order + edge-count validation also works.

2.4 Topological Sort — BFS Based (Kahn's Algorithm)

What it's for: Same goal as DFS topo sort (linear order respecting dependencies) but built using **in-degree counting + BFS** instead of recursion. Its big bonus: it naturally **detects cycles** as a side effect (if not all nodes get processed, a cycle exists).

Use Cases: - Same as DFS topo sort: build systems, course scheduling, task ordering. - Preferred when you also need **cycle detection** with zero extra code, or want to avoid recursion depth issues. - Preferred when the problem needs **multi-level batching** — e.g., “which tasks can run in parallel at each stage” (Kahn's naturally groups nodes by processing wave).

Limitations: - Only works on DAGs (a cycle just means some nodes never reach in-degree 0, and the algorithm outputs fewer than V nodes). - Requires computing in-degrees upfront (extra $O(V+E)$ prep, though same complexity class). - Doesn't give ALL possible orderings — just one, like DFS version.

Time Complexity: $O(V + E)$. **Space Complexity:** $O(V)$ — queue + in-degree array.

Approach (short): Compute in-degree of every node → push all nodes with in-degree 0 into a queue (they have no dependency left) → pop a node, add to result, decrease in-degree of its neighbors → if any neighbor's in-degree becomes 0, push it. Think “process what's ready right now, unlock the next ready batch.”

Keywords & Constraints: - Same trigger words as DFS topo sort, PLUS: “detect if scheduling is possible”, “parallel batches/levels”, “minimum number of semesters” - Constraints: $V, E \leq 10^5$ typical.

```
from collections import deque

def topo_sort_kahn(adj, V):
    """
    adj: adjacency list of a DAG
    V: number of vertices
    Returns: (order, is_valid_dag) -- order is a valid topological sort,
           is_valid_dag is False if a cycle was detected
    """
    in_degree = [0] * V
    for u in range(V):
        for v in adj[u]:
            in_degree[v] += 1          # count incoming edges for every node

    queue = deque([u for u in range(V) if in_degree[u] == 0])
    # ^ nodes with 0 in-degree have NO unmet dependency -> ready to process first

    order = []

    while queue:
        node = queue.popleft()
        order.append(node)             # safe to process now

        for neighbor in adj[node]:
            in_degree[neighbor] -= 1   # one dependency of neighbor is satisfied
            if in_degree[neighbor] == 0:
                queue.append(neighbor) # neighbor is now ready too

    is_valid_dag = (len(order) == V)
    # if fewer than V nodes got processed, some nodes never reached in-degree 0
    # -> they're stuck in a cycle

    return order, is_valid_dag
```

Practice Questions — Topological Sort (Kahn's / BFS)

Q1. Course Schedule I (Can you finish all courses?) Given prerequisites, return true/false whether all courses can be completed. *Approach:* Run Kahn's algorithm; if `len(order) != V`, a cycle exists among prerequisites, so it's impossible — Kahn's gives cycle detection for free, unlike doing DFS topo sort where you'd need a separate recursion-stack check.

Q2. Course Schedule II (Return the order) Return a valid course order, or empty if impossible. *Approach:* Kahn's algorithm directly outputs the order list; check `is_valid_dag` for feasibility.

Q3. Minimum Number of Semesters to Finish All Courses Each “wave” of Kahn's algorithm (all currently in-degree-0 nodes) can be done in parallel in one semester; find minimum semesters. *Approach:* Process the queue in batches/levels (like BFS level order) instead of one node at a time — the number of levels processed equals the minimum semesters, something DFS topo sort can't express naturally.

Q4. Alien Dictionary (BFS version) Determine character order in an alien language from sorted word list. *Approach:*

Build char-precedence graph, then Kahn's algorithm — using BFS version lets you easily detect “invalid ordering” (cycle) and also handle the “no valid order” edge case cleanly.

Q5. Parallel Course Scheduling with Time per Course Each course has a duration; find minimum time to finish all courses with unlimited parallel workers. *Approach:* Kahn's algorithm processed level-by-level, tracking `finish_time[node] = max(finish_time[predecessors]) + duration[node]` — level-based BFS structure fits this “waves of readiness” DP naturally.

Q6. Sequence Reconstruction Verify whether a sequence is the ONLY valid topological order given pairwise ordering constraints. *Approach:* Run Kahn's algorithm but check that the queue NEVER has more than 1 node at any time (multiple ready nodes = multiple possible orders = not unique).

Q7. Find All “Sink” Tasks (Tasks nothing depends on) Identify tasks that have no dependents (nothing needs to happen after them). *Approach:* After building the graph for Kahn's, tasks with out-degree 0 are sinks — a simple degree-counting extension of the same graph setup used for Kahn's.

Q8. Detect Cycle in Directed Graph (BFS Method) Just check if a directed graph has a cycle, without needing the order. *Approach:* Run Kahn's; if the processed count is less than V , a cycle exists — often preferred over DFS cycle detection when you want to avoid recursion / deep call stacks on large graphs.

Q9. Firing Order of Employees Based on Reporting Chain Given “employee reports to manager” relationships, find a valid order to notify people top-down. *Approach:* Model reporting chain as a DAG (manager → employee), Kahn's algorithm gives a valid notify order level by level.

Q10. Build Dependency Graph for Package Manager with Level Grouping Print packages that can be installed in parallel at each stage. *Approach:* Level-order variant of Kahn's algorithm (process the whole current queue as one “wave” before moving to next), directly printing each wave — this level-grouping ability is why Kahn's (BFS) is preferred over DFS topo sort for this exact class of problems.

2.5 Minimum Spanning Tree (MST) — Concept

What it's for: A Minimum Spanning Tree connects **all vertices** of a weighted, undirected, connected graph using a **subset of edges** with **minimum total weight**, with **no cycles**. It's a tree with $V-1$ edges spanning all V vertices.

Use Cases: - Network design — laying cables/pipes/roads to connect all locations at minimum cost. - Circuit design (minimum wiring). - Clustering (remove the most expensive edges of an MST to form clusters). - Approximation algorithms for problems like TSP.

Limitations: - Only defined for **connected** graphs (if disconnected, you get a Minimum Spanning **Forest** instead). - Only applies to **undirected** graphs (directed MST is a different, harder problem — “Arborescence”). - MST minimizes TOTAL weight, not the shortest path between any two specific nodes (a common misconception — MST $\neq$ shortest path tree).

Two Main Algorithms: **Prim's** (grows tree from a starting vertex) and **Kruskal's** (sorts all edges, adds smallest that doesn't form a cycle). Covered next.

Time/Space: Depends on implementation — see each algorithm below.

2.6 Prim's Algorithm

What it's for: Builds a Minimum Spanning Tree by starting at an arbitrary vertex and **greedily growing** the tree one edge at a time, always picking the cheapest edge that connects a new vertex to the tree.

Use Cases: - Network design when the graph is **dense** (many edges relative to vertices). - Real-time/incremental MST construction (you're already "inside" the tree, growing it outward — useful when the graph is given as adjacency list/matrix directly). - Cable/road network minimum cost connection problems.

Limitations: - Needs a connected graph (fails/produces incomplete tree for disconnected graphs). - With a naive array-based implementation, $O(V^2)$ — slow for sparse graphs with huge V . Use Kruskal instead for very sparse graphs. - Requires a good priority queue implementation to be efficient on sparse graphs.

Time Complexity: - With adjacency matrix + linear search: $O(V^2)$. - With adjacency list + min-heap (binary heap): $O(E \log V)$. **Space Complexity:** $O(V + E)$.

Approach (short): Start from any node, treat it as the "grown tree." Repeatedly pick the minimum-weight edge that connects a vertex INSIDE the tree to a vertex OUTSIDE the tree, add that vertex to the tree. Repeat until all vertices included. Think "the tree grows like a blob, always eating the cheapest neighbor."

Keywords & Constraints: - "minimum cost to connect all", "minimum spanning tree", "connect all cities/nodes with min cost", "dense graph" - Constraints: works well when E is large relative to V (dense); $V \leq 10^4$, $E \leq 10^5$ - 10^6 with heap version.

```
import heapq

def prims_mst(adj, V, start=0):
    """
    adj: adjacency list, adj[u] = list of (neighbor, weight)
    V: number of vertices
    start: starting vertex (arbitrary choice, doesn't change final MST weight)
    Returns: (total_weight, list_of_mst_edges)
    """
    visited = [False] * V
    min_heap = [(0, start, -1)]  # (weight, node, parent) -- 0 cost to "reach" start itself
    total_weight = 0
    mst_edges = []

    while min_heap and len(mst_edges) < V:
        weight, node, parent = heapq.heappop(min_heap)  # always grab cheapest available edge

        if visited[node]:
            continue  # skip if this vertex already joined the tree
                        # (heap may contain stale/duplicate entries)

        visited[node] = True  # officially add this vertex to the MST
        total_weight += weight
        if parent != -1:
            mst_edges.append((parent, node, weight))

        for neighbor, edge_weight in adj[node]:
            if not visited[neighbor]:
                # push every edge leading OUT of the tree to a new vertex;
                # heap will always pop the globally cheapest one next
                heapq.heappush(min_heap, (edge_weight, neighbor, node))

    return total_weight, mst_edges
```

Practice Questions — Prim's Algorithm

Q1. Minimum Cost to Connect All Cities Given cities and costs to build roads between pairs, find minimum total cost to connect all cities. *Approach:* Direct MST problem — Prim's grows the network from any one city, always picking cheapest connecting road, guaranteeing minimum total cost.

Q2. Minimum Cost to Connect Sticks/Ropes (Often solved differently, but graph version) connect all points with minimum wiring cost given pairwise costs. *Approach:* Model points as vertices, cost as edge weight — Prim's is efficient here especially if the graph is dense (all pairs have a cost, i.e., complete graph).

Q3. Optimize Water Distribution in a Village Each house can get water from a well (cost) or be connected to another house (pipe cost); minimize total cost to supply water to all houses. *Approach:* Add a virtual "well" node connected to every house with well-cost as edge weight, then run Prim's/Kruskal's for MST — turns a hybrid problem into pure MST.

Q4. Minimum Spanning Tree in a Dense Graph Given a nearly-complete graph (E close to V^2), find the MST. *Approach:*

Prim's with adjacency matrix ($O(V^2)$) is actually competitive/preferred here since E is huge — Kruskal's would need to sort a huge edge list, making Prim's the more efficient choice for dense graphs.

Q5. Network Cable Laying Problem Telecom company wants to connect all offices with minimum total cable length. *Approach:* Classic MST via Prim's — start from any office, always extend with cheapest cable to an unconnected office.

Q6. Check if Graph Has Unique MST Determine if the MST is unique (no ties in weight during MST construction that could create an alternate MST). *Approach:* Run Prim's, and whenever multiple minimum edges tie for selection, check if swapping produces an equal-cost MST — this requires careful comparison, but the core traversal is still Prim's growth logic.

Q7. Minimum Cost to Link All Fiber Points in a City Similar to cable laying, given point coordinates, cost = Euclidean distance; connect all with minimum total wiring. *Approach:* Build a complete graph with distances as weights, Prim's is ideal here since a complete graph is maximally dense — Kruskal would be slower sorting $O(V^2)$ edges.

Q8. Reduce Redundant Connections (Find Non-Tree Edges) Given a connected graph with redundant edges, find which edges are NOT part of any MST. *Approach:* Run Prim's to build the MST, then any edge not selected during the process is redundant with respect to minimum-cost connectivity.

Q9. Total Cost to Supply Electricity to All Houses Minimize total wire length connecting all houses to a power grid. *Approach:* Same MST pattern — Prim's greedily grows the connected network from the power source outward, always picking the next cheapest house to connect.

Q10. Critical and Pseudo-Critical Edges in MST Identify edges that MUST be in every MST (critical) and edges that COULD be in some MST (pseudo-critical). *Approach:* Run Prim's/Kruskal's normally to get base MST weight, then for each edge: force-exclude it and recompute MST weight (critical if weight increases or disconnects), force-include it and recompute (pseudo-critical if weight stays equal to base) — repeated MST computation, so Prim's efficiency matters on larger graphs.

2.7 Kruskal's Algorithm

What it's for: Builds a Minimum Spanning Tree by **sorting all edges** by weight and greedily adding each edge if it doesn't form a cycle, using a **Union-Find (Disjoint Set)** structure to detect cycles instantly.

Use Cases: - MST construction on **sparse** graphs (few edges). - When the graph is naturally given as an **edge list** rather than adjacency list. - Clustering algorithms (stop early after k merges to get k clusters). - Network design problems where edges (not adjacency) are the natural unit of data.

Limitations: - Requires sorting all edges upfront: $O(E \log E)$ — can be slower than Prim's on very dense graphs. - Needs a Union-Find structure to be efficient (without it, cycle checking is slow). - Like Prim's, only works for undirected, connected graphs (for MST; disconnected gives MST of each component / forest).

Time Complexity: $O(E \log E)$ for sorting + $O(E \alpha(V))$ for union-find operations (α = inverse Ackermann, practically constant) → overall $O(E \log E)$. **Space Complexity:** $O(V + E)$.

Approach (short): Sort all edges by weight ascending. Go through them one by one — add an edge to the MST only if its two endpoints are in **different** components (use Union-Find to check/merge). Stop when you've added $V-1$ edges. Think "always take the cheapest available edge, unless it would create a cycle."

Keywords & Constraints: - Same MST trigger words as Prim's, PLUS: "given as list of edges", "sparse graph", "minimize cost with no cycles" - Constraints: $E \leq 10^5 - 10^6$, works great when E is much smaller than V^2 .

```
def kruskals_mst(edges, V):
    """
    edges: list of (weight, u, v) tuples
    V: number of vertices
    Returns: (total_weight, list_of_mst_edges)
    """
    parent = list(range(V)) # each node starts as its own component representative
    rank = [0] * V          # rank = rough "tree depth" used to keep union-find flat

    def find(x):
        # path compression: point x directly to the root so future finds are faster
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        root_x, root_y = find(x), find(y)
        if root_x == root_y:
            return False # already in same component -> would form a cycle
        # union by rank: attach smaller tree under bigger tree to keep it flat
        if rank[root_x] < rank[root_y]:
            root_x, root_y = root_y, root_x
        parent[root_y] = root_x
        if rank[root_x] == rank[root_y]:
            rank[root_x] += 1
        return True

    edges.sort(key=lambda e: e[0]) # always consider the cheapest edge first

    total_weight = 0
    mst_edges = []

    for weight, u, v in edges:
        if union(u, v): # only add edge if it connects 2 different components
            total_weight += weight
            mst_edges.append((u, v, weight))
            if len(mst_edges) == V - 1:
                break # MST complete, stop early

    return total_weight, mst_edges
```

Practice Questions — Kruskal's Algorithm

Q1. Minimum Cost to Connect All Points (Sparse Edge List Given) Given an explicit list of possible connections and costs, find minimum cost to connect everything. *Approach:* Since the input is already an edge list, Kruskal's is the natural fit — sort edges, union-find to avoid cycles.

Q2. Number of Operations to Make Network Connected Given a network with redundant connections, find minimum cable removals/additions to connect all computers. *Approach:* Use Union-Find (Kruskal's core mechanism) to count connected components and redundant edges — if redundant edges $\geq$ components needed to merge, it's possible.

Q3. Connecting Cities with Minimum Cost (with some forbidden connections) Some city pairs can't be connected directly; find minimum total cost to connect all cities. *Approach:* Build edge list only from allowed connections, run Kruskal's — naturally skips forbidden pairs since they're just absent from the edge list.

Q4. Redundant Connection (Find the Extra Edge Causing a Cycle) Given a graph that was a tree plus one extra edge, find that extra edge. *Approach:* Process edges one by one with Union-Find; the first edge where `union()` returns False (both endpoints already connected) is the redundant one — this is literally Kruskal's cycle-check in isolation.

Q5. Minimum Spanning Forest for Disconnected Graph Graph isn't fully connected; find MST for each component. *Approach:* Run Kruskal's normally — it naturally stops adding edges once no more valid (non-cycle) edges exist per component; total edges added will be $V - (\text{number of components})$.

Q6. Least Cost to Connect All Islands with Bridges Given islands and bridge-building costs between pairs, connect all islands at minimum cost. *Approach:* Straightforward MST via Kruskal's, especially natural since "bridge options" are naturally given as an edge list with costs.

Q7. Clustering with K Clusters (Max Spacing) Given points and distances, split into K clusters maximizing the minimum distance between clusters. *Approach:* Run Kruskal's but STOP early after $V - K$ merges instead of $V - 1$ — leftover unmerged components are the K clusters; this early-stop trick is a well-known Kruskal's variant.

Q8. Check if Adding an Edge Creates a Cycle Given a graph and a new edge, determine if adding it creates a cycle. *Approach:* Union-Find check: if `find(u) == find(v)` already, adding edge `(u,v)` creates a cycle — same cycle-check logic Kruskal's uses internally.

Q9. Optimize Sparse Network with Many Isolated Edge Options Very large V but each vertex only has a few possible connections (sparse edge list). *Approach:* Kruskal's $O(E \log E)$ beats Prim's $O(V^2)$ here since $E \ll V^2$ — sorting a small edge list is cheap and union-find operations are near constant time.

Q10. Second-Best Minimum Spanning Tree Find the MST with the second-smallest total weight (must differ from the actual MST by at least one edge). *Approach:* First build MST via Kruskal's, then for each edge NOT in the MST, try swapping it in (replacing the max-weight edge on the cycle it creates) and track the minimum resulting total — builds directly on top of the standard Kruskal's construction.

2.8 Union-Find (Disjoint Set Union - DSU)

What it's for: A data structure (not exactly a “traversal” algorithm) that efficiently tracks a collection of disjoint (non-overlapping) sets, and answers two questions fast: “are these two elements in the same set?” (`find`) and “merge these two sets” (`union`). It's the engine behind Kruskal's algorithm and many connectivity problems.

Use Cases: - Cycle detection while building a graph edge by edge (Kruskal's). - Counting/tracking connected components dynamically as edges are added. - Detecting redundant connections. - Network connectivity queries (“are these 2 computers on the same network?”). - Percolation problems, image processing (connected regions), account/friend merging problems.

Limitations: - Doesn't store the actual structure of the graph (no edges/paths) — only “which set am I in,” so you can't reconstruct a path from it. - Standard version doesn't support “un-union”/split operations efficiently. - Without both **path compression** and **union by rank/size**, worst-case can degrade to $O(V)$ per operation (a long chain).

Time Complexity: With path compression + union by rank: $O(\alpha(V))$ per operation — practically $O(1)$ (α = inverse Ackermann function, grows extremely slowly, ≤ 4 for any realistic input). **Space Complexity:** $O(V)$.

Approach (short): Each element starts as its own parent (its own set). `find(x)` walks up parent pointers to the root, compressing the path along the way (make every visited node point straight to root). `union(x, y)` finds both roots and attaches the smaller/shallower tree under the bigger one. Think “everyone points to their group leader; when merging groups, the smaller group joins the bigger group's leader.”

Keywords & Constraints: - “same group/network/component”, “merge”, “redundant connection”, “dynamic connectivity”, “friend circles” - Constraints: extremely fast, so $V, E \leq 10^6$ is comfortable.

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n)) # every node is its own parent (own set) initially
        self.rank = [0] * n         # rank ~ upper bound on tree height, for balancing
        self.count = n              # number of disjoint sets (components), decreases on union

    def find(self, x):
        if self.parent[x] != x:
            # PATH COMPRESSION: recursively find root, then re-point x directly to it.
            # This flattens the tree so future find() calls are near O(1).
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)

        if root_x == root_y:
            return False # already in the same set -> nothing to do (cycle if edge)

        # UNION BY RANK: attach the shorter tree under the taller tree's root
        # to avoid creating long chains that slow down future find() calls.
        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1

        self.count -= 1 # two sets merged into one
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y) # same root -> same set
```

Practice Questions — Union-Find

Q1. Number of Provinces (Friend Circles) Given a matrix of direct connections, count total number of groups (provinces). *Approach:* Union every directly connected pair; the final `UnionFind.count` gives the number of groups — avoids writing a separate BFS/DFS traversal.

Q2. Redundant Connection Find the one edge that, if removed, turns the graph back into a tree (i.e., the edge causing the cycle). *Approach:* Process edges in given order, union each; the first edge where `union()` returns False is the answer — union-find in its purest cycle-detection form.

Q3. Accounts Merge Merge user accounts that share at least one common email. *Approach:* Treat each account as a node, union accounts sharing an email — union-find efficiently groups all transitively-connected accounts without explicit graph traversal.

Q4. Number of Islands II (Dynamic/Online) Islands are added one at a time (land cells appear over time); after each addition, report current number of islands. *Approach:* Union-Find is ideal here because it supports incremental merging in near- $O(1)$ per operation — re-running BFS/DFS after every single addition would be far too slow.

Q5. Most Stones Removed with Same Row or Column Stones sharing a row or column can be removed (keep one representative); find max stones removable. *Approach:* Union stones sharing row/column; answer = `total_stones - number_of_components`, since one stone per component must remain — union-find easily counts components without needing the actual graph edges.

Q6. Satisfiability of Equality Equations Given equations like `a==b` and `a!=b`, determine if they can all be simultaneously true. *Approach:* Union all variables connected by `==`; then check every `!=` equation — if both sides are in the same set, it's a contradiction. Union-Find naturally groups “must be equal” variables.

Q7. Smallest String With Swaps Given pairs of indices that can be freely swapped, find the lexicographically smallest string achievable. *Approach:* Union all indices connected by swap-pairs into groups; characters within a group can be freely rearranged, so sort characters within each group — union-find identifies which positions belong together.

Q8. Graph Valid Tree Given `n` nodes and a list of edges, determine if they form a valid tree. *Approach:* A valid tree needs exactly `n-1` edges AND no cycles AND full connectivity — union-find checks both: if any `union()` fails (cycle) it's invalid, and final `count == 1` confirms full connectivity.

Q9. Minimize Malware Spread Given a network and infected nodes, determine which infected node's removal minimizes total spread. *Approach:* Union-Find groups nodes into connected components first; then analyze how many infected nodes fall in each component to decide which removal has the biggest isolated impact — union-find gives fast component membership lookups.

Q10. Evaluate Division (Graph with Weighted Union-Find or DFS) Given equations like `a/b = 2.0`, answer queries like `a/c = ?`. *Approach:* Usually solved with weighted-edge DFS/BFS, but can also be solved with a **weighted union-find** where each node stores a ratio to its parent — included here to show union-find's extension beyond simple connectivity into ratio/relationship tracking.

2.9 Dijkstra's Algorithm

What it's for: Finds the **shortest path from a single source** to all other vertices in a **weighted graph with non-negative edge weights**. The “go-to” shortest path algorithm when weights are all ≥ 0 .

Use Cases: - GPS navigation / shortest route with distances or travel times. - Network routing protocols (OSPF uses Dijkstra-like logic). - Flight/train ticket cost minimization. - Any “minimum cost/time to reach” problem where costs are non-negative.

Limitations: - **Fails with negative edge weights** — can produce wrong (too-large) shortest paths because once a node is finalized, Dijkstra never revisits it, but a later negative edge could have offered a cheaper path. - Doesn't detect negative cycles at all (use Bellman-Ford for that). - Single-source only — to get all-pairs shortest paths you'd run it V times ($O(V * E \log V)$), or just use Floyd-Warshall directly if V is small.

Time Complexity: $O((V + E) \log V)$ with a binary heap (min-heap) priority queue. **Space Complexity:** $O(V + E)$.

Approach (short): Like BFS, but instead of a plain queue, use a **min-heap** ordered by current shortest distance found so far. Always expand the closest unvisited node next. When you pop a node, its distance is now guaranteed final (because non-negative weights mean nothing pops later could ever be smaller). Think “BFS, but always process the currently cheapest node instead of FIFO order.”

Keywords & Constraints: - “shortest path”, “minimum cost”, “minimum time”, “non-negative weights”, “single source shortest path” - Constraints: $V \leq 10^5$, $E \leq 10^5$ - 10^6 with heap implementation — this is the default choice unless negatives are explicitly mentioned.

```
import heapq

def dijkstra(adj, source, V):
    """
    adj: adjacency list, adj[u] = list of (neighbor, weight); weights must be >= 0
    source: starting vertex
    V: number of vertices
    Returns: list of shortest distances from source to every vertex
    """
    dist = [float('inf')] * V    # start assuming everything is unreachable
    dist[source] = 0             # distance to itself is always 0

    min_heap = [(0, source)]    # (distance_so_far, node) -- heap always gives smallest first

    while min_heap:
        current_dist, node = heapq.heappop(min_heap)    # pick the closest unfinalized node

        if current_dist > dist[node]:
            continue    # stale entry (we already found a better path), skip it

        for neighbor, weight in adj[node]:
            new_dist = current_dist + weight    # try going through 'node'

            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist    # found a shorter path to neighbor
                heapq.heappush(min_heap, (new_dist, neighbor))    # relax the edge: update best known dist

    return dist
```

Practice Questions — Dijkstra's Algorithm

Q1. Network Delay Time Given a network with signal travel times between nodes, find time for a signal from source to reach all nodes. *Approach:* Direct single-source shortest path with non-negative weights — textbook Dijkstra application.

Q2. Cheapest Flights Within K Stops (variation) Find cheapest flight cost with at most K stops. *Approach:* Modified Dijkstra (or Bellman-Ford) tracking (cost, stops) state — plain Dijkstra alone doesn't respect the “ K stops” constraint since it only tracks minimum cost, not path length, so you must augment the state.

Q3. Path With Minimum Effort Find a path where the maximum absolute height difference along the path is minimized. *Approach:* “Dijkstra-style” traversal but relaxing on $\max(\text{current_max_effort}, \text{edge_diff})$ instead of sum — shows Dijkstra's greedy priority-queue pattern generalizes beyond pure sums to any “minimize the worst step” metric.

Q4. Swim in Rising Water Find minimum time to swim from top-left to bottom-right of a grid where cell values are elevations. *Approach:* Same “minimize the maximum value along path” pattern as effort problem — solved with a priority queue exactly like Dijkstra, just with a different relaxation rule.

Q5. Shortest Path with Alternating Colors / Constraints Find shortest path where edges must alternate between two types/colors. *Approach:* Dijkstra/BFS on an augmented state graph $(\text{node}, \text{last_edge_color})$ instead of just node — shows

how Dijkstra generalizes when the “state” needs more than just the current vertex.

Q6. Path with Maximum Probability Find the path between two nodes that maximizes the product of edge probabilities. *Approach:* Dijkstra but with a MAX-heap and multiplying probabilities instead of adding weights (since probabilities are between 0-1, and Dijkstra’s greedy “process best first” logic still holds because probability only decreases as you add edges — same monotonic property that makes Dijkstra correct for sums).

Q7. Minimum Cost to Reach Destination in Time Find minimum cost path from source to destination that also respects a time budget. *Approach:* Modified Dijkstra/DP with state `(node, time_used)` — since we now have 2 constraints (cost + time), plain single-value Dijkstra distance array isn’t enough; needs a distance table indexed by time too.

Q8. Dijkstra on Grid With Weighted Cells Find minimum cost path through a grid where each cell has a traversal cost. *Approach:* Treat each grid cell as a graph node with edges to its neighbors weighted by the neighbor’s cost — standard Dijkstra once the grid is reframed as a graph.

Q9. Second Shortest Path Find the length of the strictly second shortest path (not equal to the shortest) from source to destination. *Approach:* Track top-2 distinct shortest distances per node while relaxing edges (instead of just 1) — an augmented Dijkstra that keeps 2 best values per node rather than just the minimum.

Q10. Shortest Path in a Graph with Time-Dependent / Delayed Edges Some edges can only be “used” during specific time windows (like traffic signals). *Approach:* Dijkstra where the relaxation step computes “wait until edge becomes usable, then add travel time” — same greedy priority-queue framework, just a smarter edge-weight calculation function.

2.10 Bellman-Ford Algorithm

What it's for: Finds shortest paths from a single source to all vertices, and **works correctly with negative edge weights**. It can also **detect negative weight cycles** (a cycle whose total weight is negative — shortest path becomes undefined/ `-infinity` if one exists and is reachable).

Use Cases: - Graphs where edges can have negative weights (e.g., currency arbitrage: detect if a sequence of currency conversions yields free profit = negative cycle). - Detecting negative cycles specifically (fraud/arbitrage detection). - Distance-vector routing protocols (e.g., RIP uses Bellman-Ford-like updates). - When you need correctness guarantees over pure speed (Dijkstra is faster but breaks with negatives).

Limitations: - Much **slower** than Dijkstra: $O(V * E)$ vs $O(E \log V)$. - Still doesn't give a meaningful "shortest path" if a negative cycle is reachable from the source and can reach the destination — distance becomes `-infinity` (undefined), it can only detect and report this. - Overkill (unnecessarily slow) for graphs known to have only non-negative weights — use Dijkstra there instead.

Time Complexity: $O(V * E)$. **Space Complexity:** $O(V)$.

Approach (short): Relax **all edges**, $V-1$ times. Why $V-1$? Because the longest possible shortest path (without cycles) uses at most $V-1$ edges, so after $V-1$ full relaxation rounds, all shortest distances are guaranteed final. Do **one more round**: if any distance can still be improved, a negative cycle exists. Think "brute-force relax everything, over and over, enough times to guarantee correctness even with negative weights."

Keywords & Constraints: - "negative weights", "negative cycle", "arbitrage", "currency conversion", "at most k edges/stops" - Constraints: $V \leq 10^3$ - 10^4 , $E \leq 10^4$ - 10^5 (since it's $O(V * E)$, too slow for very large graphs).

```
def bellman_ford(edges, source, V):
    """
    edges: list of (u, v, weight) -- weight can be negative
    source: starting vertex
    V: number of vertices
    Returns: (dist, has_negative_cycle)
    """
    dist = [float('inf')] * V
    dist[source] = 0

    # Relax all edges V-1 times.
    # After k iterations, dist[] is guaranteed correct for all shortest paths
    # using at most k edges. V-1 iterations covers every possible simple path length.
    for _ in range(V - 1):
        updated = False
        for u, v, weight in edges:
            if dist[u] != float('inf') and dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight # found a cheaper way to reach v via u
                updated = True
        if not updated:
            break # no changes this round means it has already converged, stop early

    # One extra pass: if anything STILL improves, a negative-weight cycle exists,
    # because a genuine shortest path should never need more than V-1 edges.
    has_negative_cycle = False
    for u, v, weight in edges:
        if dist[u] != float('inf') and dist[u] + weight < dist[v]:
            has_negative_cycle = True
            break

    return dist, has_negative_cycle
```

Practice Questions — Bellman-Ford Algorithm

Q1. Cheapest Flights Within K Stops Find the cheapest price from source to destination using at most K stops. *Approach:* Run a Bellman-Ford-style relaxation limited to exactly $K+1$ rounds (not full $V-1$) — this naturally caps path length to K edges, something Dijkstra can't do without extra state tracking.

Q2. Detect Arbitrage Opportunity in Currency Exchange Given exchange rates between currencies, detect if a sequence of trades yields guaranteed profit. *Approach:* Convert rates to $-\log(\text{rate})$ edge weights (so multiplying rates becomes adding logs); a negative cycle in this transformed graph = an arbitrage opportunity — the classic real-world reason Bellman-Ford's negative cycle detection matters.

Q3. Shortest Path with Negative Edge Weights Plain shortest path problem where some edges are explicitly negative. *Approach:* Must use Bellman-Ford — Dijkstra would give an incorrect (too high) answer because it finalizes nodes greedily and never revisits them even if a later negative edge offers a cheaper route.

Q4. Find if Negative Cycle Exists in Graph Just answer yes/no whether the graph has a negative weight cycle reachable from a given source. *Approach:* Run Bellman-Ford's extra v -th relaxation round; if any edge still relaxes successfully, a negative cycle exists — this is literally Bellman-Ford's built-in detection step used standalone.

Q5. Currency Conversion Chain with Minimum Steps Find minimum number of currency conversions (steps) to get the best rate. *Approach:* Track both cost and number of edges used in the state during Bellman-Ford relaxation to also minimize steps among equally-optimal-cost paths.

Q6. Shortest Path in a Graph with Some Negative but No Negative Cycles Confirmed no negative cycles exist; find all shortest paths from source. *Approach:* Standard Bellman-Ford without needing the extra detection round (though it's cheap insurance to keep it) — negatives alone don't break Bellman-Ford, only negative CYCLES do.

Q7. Find Path with Exactly K Edges and Minimum Cost Find minimum cost path from source to destination using EXACTLY K edges (not at most). *Approach:* DP variant of Bellman-Ford: $\text{dist}[k][v] = \text{min cost to reach } v \text{ using exactly } k \text{ edges}$, relaxed edge-by-edge for exactly K rounds — Bellman-Ford's edge-relaxation structure adapts naturally to "exact edge count" DP.

Q8. Detect if a Negative Cycle Affects the Destination Confirm a negative cycle exists AND is actually reachable from source AND can reach the destination (some negative cycles are irrelevant if unreachable). *Approach:* Run Bellman-Ford from source to find reachable negative-cycle nodes, then run Bellman-Ford (or BFS) on the REVERSED graph from destination to check which of those nodes can also reach the destination — combines Bellman-Ford with reachability analysis.

Q9. Foreign Exchange Rate Optimal Conversion Path Similar to arbitrage, but instead find the actual best conversion PATH (not just detect a cycle) between two specific currencies. *Approach:* Standard Bellman-Ford shortest path (after log-transforming rates) with path reconstruction via parent pointers.

Q10. Distance Vector Routing Simulation Simulate how routers using distance-vector protocols converge on shortest paths, handling links that can temporarily have "negative-like" cost corrections. *Approach:* Bellman-Ford's iterative, all-edges-relaxed-repeatedly nature mirrors exactly how distance-vector routing protocols propagate and converge on shortest paths across a network.

2.11 Floyd-Warshall Algorithm

What it's for: Finds the shortest paths between **every pair of vertices** (all-pairs shortest path) in a weighted graph, and works with negative edge weights (but not negative cycles reachable within the path).

Use Cases: - Small/medium dense graphs where you need distances between ALL pairs, not just from one source. - Precomputing a distance table for repeated queries ("what's the shortest distance between any 2 of these 200 cities" asked many times). - Detecting negative cycles (if $\text{dist}[i][i] < 0$ for any i after running, a negative cycle exists). - Transitive closure computation (reachability between all pairs) as a boolean variant.

Limitations: - $O(V^3)$ time — too slow for large graphs ($V > \sim 500$ becomes risky, $V > 1000$ usually too slow). - $O(V^2)$ space for the distance matrix — memory-heavy for large V . - Overkill if you only need shortest path from ONE source (use Dijkstra/Bellman-Ford instead — they're faster for that case).

Time Complexity: $O(V^3)$. **Space Complexity:** $O(V^2)$.

Approach (short): Try every vertex k as a possible **intermediate** stop for every pair (i, j) . If going $i \rightarrow k \rightarrow j$ is cheaper than the current known $i \rightarrow j$, update it. After trying all vertices as intermediates, the matrix holds true shortest paths between all pairs. Think "for every possible middle stop, check if detouring through it makes any route cheaper — do this for all routes and all possible middle stops."

Keywords & Constraints: - "all pairs shortest path", "shortest distance between every pair", "small number of vertices", "transitive closure" - Constraints: $V \leq 400-500$ typically (cubic time), often explicitly small in problem statements.

```
def floyd_warshall(V, edges):
    """
    V: number of vertices
    edges: list of (u, v, weight) -- directed edges (for undirected, add both directions)
    Returns: dist matrix where dist[i][j] = shortest distance from i to j
            (or float('inf') if unreachable)
    """
    # Initialize distance matrix: 0 to self, edge weight if direct edge exists, else infinity
    dist = [[float('inf')] * V for _ in range(V)]
    for i in range(V):
        dist[i][i] = 0  # distance from a node to itself is 0

    for u, v, weight in edges:
        dist[u][v] = min(dist[u][v], weight)  # handle possible duplicate edges, keep cheapest

    # Core triple loop: k = intermediate node, i = source, j = destination
    for k in range(V):
        for i in range(V):
            for j in range(V):
                # if going i -> k -> j is cheaper than the current known i -> j, update it
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    # Negative cycle check: if any node's distance to ITSELF became negative,
    # it means there's a negative cycle passing through it.
    has_negative_cycle = any(dist[i][i] < 0 for i in range(V))

    return dist, has_negative_cycle
```

Practice Questions — Floyd-Warshall Algorithm

Q1. Find the City With the Smallest Number of Neighbors at a Threshold Distance Given city connections with weights, find shortest distances between all pairs, then answer a threshold-based city ranking query. *Approach:* Since we need ALL pairwise shortest distances (not just from one city) and V is small, Floyd-Warshall is the natural fit — running Dijkstra V times would also work but Floyd-Warshall's simplicity wins for small V .

Q2. All Pairs Shortest Path in a Small Dense Graph Given a small graph, output the full distance matrix between all vertex pairs. *Approach:* Direct Floyd-Warshall application — this is exactly the problem it's designed for.

Q3. Detect Negative Cycle in the Graph Determine if any negative weight cycle exists anywhere in the graph. *Approach:* Run Floyd-Warshall, then check if any $\text{dist}[i][i] < 0$ — a diagonal going negative directly signals a cycle back to the same node with negative total weight.

Q4. Transitive Closure of a Graph Determine for every pair (i, j) whether j is reachable from i at all (boolean, not distance). *Approach:* Floyd-Warshall variant using OR/AND boolean logic instead of min/sum: $\text{reach}[i][j] = \text{reach}[i][j] \text{ OR } (\text{reach}[i][k] \text{ AND } \text{reach}[k][j])$ — same triple-loop structure, different combination operator.

Q5. Find the Town Judge / Central Hub Using Distance Sums Find the vertex with the minimum total distance to all other vertices (graph centroid problem). *Approach:* Run Floyd-Warshall to get all pairwise distances, then sum each row

and pick the minimum — needing full pairwise data makes this a natural Floyd-Warshall use case for small graphs.

Q6. Currency Conversion Table (All Pairs Best Rate) Given exchange rates between currency pairs, compute the best possible conversion rate between every pair of currencies (through any chain of conversions). *Approach:* Floyd-Warshall variant using multiplication and max instead of addition and min: `rate[i][j] = max(rate[i][j], rate[i][k] * rate[k][j])` — shows the algorithm's core triple-loop pattern generalizes to other "combine via intermediate" problems.

Q7. Minimum Number of Edges to Reverse for Full Connectivity Given a directed graph, find minimum edge reversals needed to make it possible to reach every vertex from a given source. *Approach:* Not the primary tool, but Floyd-Warshall style all-pairs reachability precomputation helps evaluate multiple candidate reversal scenarios efficiently on small graphs.

Q8. Shortest Path with At Most One Edge Removed (Small Graph) Given a small graph, find how removing each single edge affects shortest paths between all pairs. *Approach:* Precompute all-pairs shortest paths with Floyd-Warshall once, then for each edge removal, only recompute affected paths — much faster than rerunning full shortest-path algorithms from scratch for every removal.

Q9. Find if Graph Can Be Divided Into Two Sets With Bounded Max Distance Given all-pairs distances, determine if vertices can be grouped such that max intra-group distance $\leq$ threshold. *Approach:* Compute full distance matrix via Floyd-Warshall first (needed as a base for the grouping logic), then apply threshold-based clustering/checking on top.

Q10. Evaluate Division (All Pairs Version) Given `a/b = k` type equations, answer ALL possible pairwise division queries efficiently (as opposed to just a few queries, which favors DFS per query). *Approach:* Floyd-Warshall variant with multiplication: `ratio[i][j] = ratio[i][k] * ratio[k][j]` — precomputing via Floyd-Warshall makes sense when there are MANY queries on a SMALL variable set; for just a few queries, per-query DFS is simpler and avoids unnecessary $O(V^3)$ work.

PART 3: QUICK REFERENCE CHEAT SHEET

Algorithm	Purpose	Time Complexity	Space	Fails When
BFS	Shortest path (unweighted)	$O(V+E)$	$O(V)$	Weighted graphs
DFS	Traversal, cycle/component detection	$O(V+E)$	$O(V)$	Needs shortest path
Topo Sort (DFS)	Ordering with dependencies	$O(V+E)$	$O(V)$	Graph has a cycle
Topo Sort (Kahn's/BFS)	Ordering + cycle detection	$O(V+E)$	$O(V)$	Graph has a cycle
Prim's	MST (dense graphs)	$O(E \log V)$	$O(V+E)$	Disconnected graph
Kruskal's	MST (sparse graphs)	$O(E \log E)$	$O(V+E)$	Disconnected graph (gives forest)
Union-Find	Connectivity / grouping	$O(\alpha(V)) \sim O(1)$	$O(V)$	Needs actual path info
Dijkstra	Shortest path (single source)	$O((V+E) \log V)$	$O(V+E)$	Negative weights
Bellman-Ford	Shortest path + negative weights	$O(V \cdot E)$	$O(V)$	Very large graphs (slow)
Floyd-Warshall	All-pairs shortest path	$O(V^3)$	$O(V^2)$	Large V ($V > \sim 500$)

Decision Guide — “Which Algorithm Do I Use?”

1. **Need shortest path, all edges equal weight?** → **BFS**
2. **Need to explore/detect cycles/components, order doesn't matter?** → **DFS**
3. **Need a valid order respecting “A before B” dependencies?** → **Topological Sort** (Kahn's if you also want cycle detection or level-batching, DFS version if simpler/recursive is fine)
4. **Need to connect all nodes at minimum total cost, graph is dense?** → **Prim's**
5. **Need to connect all nodes at minimum total cost, edges given as list / graph is sparse?** → **Kruskal's** (+ **Union-Find**)
6. **Need fast “same group?” or dynamic merging queries?** → **Union-Find**
7. **Need shortest path from ONE source, all weights ≥ 0 ?** → **Dijkstra**
8. **Need shortest path from ONE source, negative weights possible, or need to detect negative cycles?** → **Bellman-Ford**
9. **Need shortest path between EVERY pair of nodes, graph is small?** → **Floyd-Warshall**